

CHAPTER-1

INTRODUCTION

Coal mining remains one of the most dangerous occupations worldwide. Workers are constantly exposed to hazardous gases such as methane (CH₄) and carbon monoxide (CO), oxygen-deficient environments, high temperatures, humidity, and underground pressure variations. In addition, accidents such as roof collapses, falls, and sudden health issues increase the risk of fatal injuries. Traditional safety systems in coal mines primarily focus on environmental monitoring at fixed locations and do not continuously track individual worker health conditions. This creates a delay in emergency detection and rescue operations. To overcome these limitations, this project proposes a wearable IoT-based safety monitoring system integrated into a smart helmet and jacket. The helmet includes environmental and motion sensors such as DHT11 (temperature and humidity), gas sensors, and an IMU angle sensor. The jacket integrates health-monitoring sensors, including SpO₂, body temperature, and pressure. All collected data is transmitted in real time to a monitoring station through IoT technology, ensuring immediate detection of unsafe conditions and faster emergency response. This solution enhances worker safety by enabling early detection of health deterioration or hazardous environmental changes, allowing for timely alerts and intervention. Its low cost, scalability, and reliability make it highly suitable for deployment in large-scale mining operations with minimal infrastructure.

Keywords: Smart jacket, Coal Mining, Wearable jacket, Mine Worker Safety, Embedded system, Microcontroller, Smart Safety Equipment

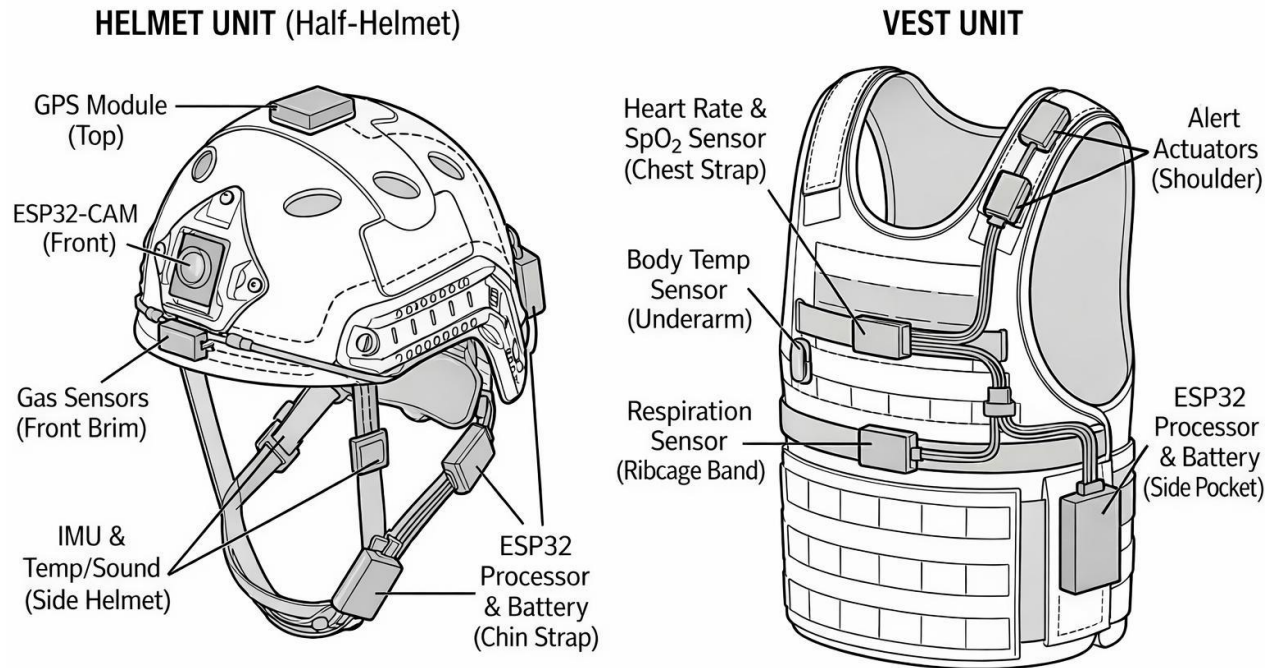


Fig 1.1 Physical layout of the prototype hardware modules integrated into the helmet and vest

1.1 EMBEDDED SYSTEMS

1.1.1 Overview of embedded systems

An embedded system is a special-purpose computer system designed to perform one or a few dedicated functions, often with real-time computing constraints. It is usually embedded as part of a complete device including hardware and mechanical parts. In contrast, a general-purpose computer, such as a personal computer, can do many different tasks depending on programming. Embedded systems have become very important today as they control many of the common devices we use. Since the embedded system is dedicated to specific tasks, design engineers can optimize it, reducing the size and cost of the product, or increasing the reliability and performance. Some embedded systems are mass-produced, benefiting from economies of scale. Embedded systems range from portable devices such as digital watches and MP3 players, to large stationary installations like traffic lights, factory controllers, or the systems controlling nuclear power plants. Complexity varies from low, with a single microcontroller chip, to very high with multiple units, peripherals and networks mounted inside a large chassis or enclosure. In general, "embedded system" is not an exactly defined term, as many systems have some element of programmability. For example,

Handheld computers share some elements with embedded systems such as the operating systems and microprocessors which power them but are not truly embedded systems, because they allow different applications to be loaded and peripherals to be connected.

1.1.2 Embedded systems provide several functions

- Monitor the environment; embedded systems read data from input sensors. This data is then processed and the results displayed in some format to a user or users
- Control the environment; embedded systems generate and transmit commands for actuators.
- Transform the information; embedded systems transform the data collected in some meaningful way, such as data compression/decompression although interaction with the external world via sensors and actuators is an important aspect of embedded systems, these systems also provide functionality specific to their applications. Embedded systems typically execute applications such as control laws, finite state machines, and signal processing algorithms. These systems must also detect and react to faults in both the internal computing environment as well as the surrounding electromechanical systems. There are many categories of embedded systems, from communication devices to home appliances to control systems. Examples include;
 - Communication devices
e.g.: modems, cellular phones
 - Home Appliances
e.g.: CD player, VCR, microwave oven
 - Control Systems
e.g.: Automobile anti-lock braking systems, robotics, satellite control

1.2 Block diagram of an embedded system:

An embedded system usually contains an embedded processor. Many appliances that have a digital interface microwave, VCRs, cars utilize embedded systems. Some embedded systems include an operating system. Others are very specialized resulting in the entire logic being implemented as a single program. These systems are embedded into some device for some specific purpose other than to provide general purpose computing.

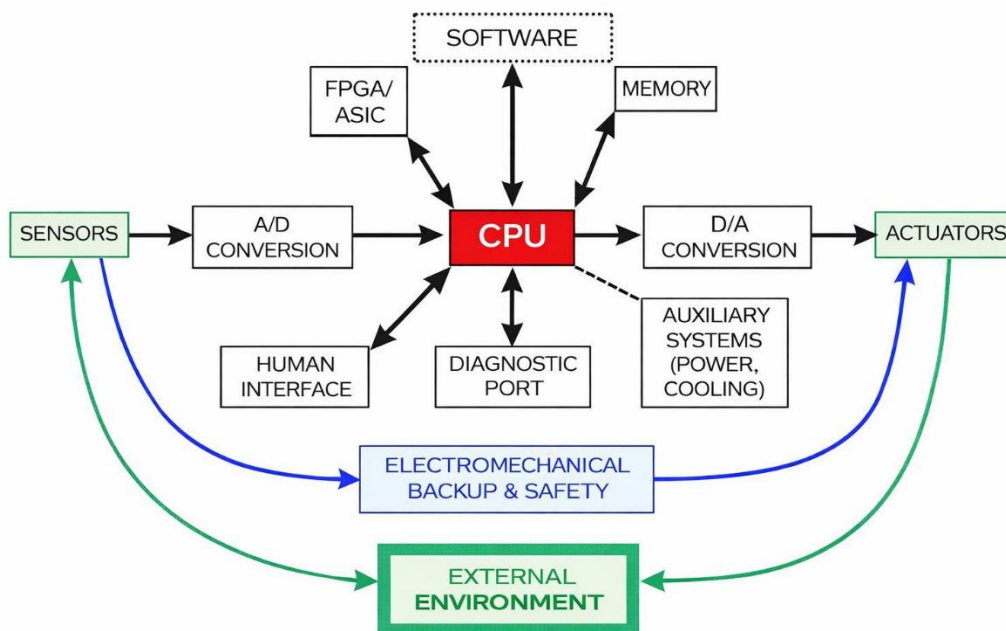


Fig 1.2 Block diagram of a typical embedded system

1.3 Characteristics of embedded systems

Embedded systems are characterized by a unique set of characteristics. Each of these characteristics imposed a specific set of design constraints on embedded systems designers. The challenge to designing embedded systems is to conform to the specific set of constraints for the application.

1.4 Application Specific Systems:

Embedded systems are not general-purpose computers. Embedded system designs are optimized for a specific application. Many of the job characteristics are known before the hardware is designed. This allows the designer to focus on the specific design constraints of a well-defined application. As such, there is limited user reprogram ability.

Some embedded systems, however, require the flexibility of reprogram ability. Programmable DSPs are common for such applications.

1.5 Reactive Systems

As mentioned earlier, a typical embedded systems model responds to the environment via sensors and control the environment using actuators. This requires embedded systems to run at the speed of the environment. This characteristic of embedded system is called “reactive”. Reactive computation means that the system (primarily the software component) executes in response to external events. External events can be either periodic or aperiodic. Periodic events make it easier to schedule processing to guarantee performance. Aperiodic events are harder to schedule. The maximum event arrival rate must be estimated in order to accommodate worst case situations. Most embedded systems have a significant reactive component. One of the biggest challenges for embedded system designers is performing an accurate worst case design analysis on systems with statistical performance characteristics (e.g., cache memory on a DSP or another embedded processor). Real time system operation means that the correctness of a computation depends, in part, on the time at which it is delivered. Systems with this requirement must often design to worst case performance. But accurately predicting the worst case may be difficult on complicated architectures. This often leads to overly pessimistic estimates erring on the side of caution. Many embedded systems have a significant requirement for real time operation in order to meet external I/O and control stability requirements. Many real-time systems are also reactive systems.

1.6 Distributed Systems

A common characteristic of an embedded system is one that consists of communicating processes executing on several CPUs or ASICs which are connected by communication links. The reason for this is economy. Economical 4 8-bit microcontrollers may be cheaper than 32-bit processors. Even after adding the cost of the communication links, this approach may be preferable. In this approach, multiple processors are usually required to handle multiple time-critical tasks. Devices under control of embedded systems may also be physically distributed.

1.7 Heterogeneous Architectures

Embedded systems often are composed of heterogeneous architectures (Fig 1.2). They may contain different processors in the same system solution. They may also be mixed signal systems. The combination of I/O interfaces, local and remote memories, and sensors and actuators make embedded system design truly unique. Embedded systems also have tight design constraints, and heterogeneity provides better design flexibility.

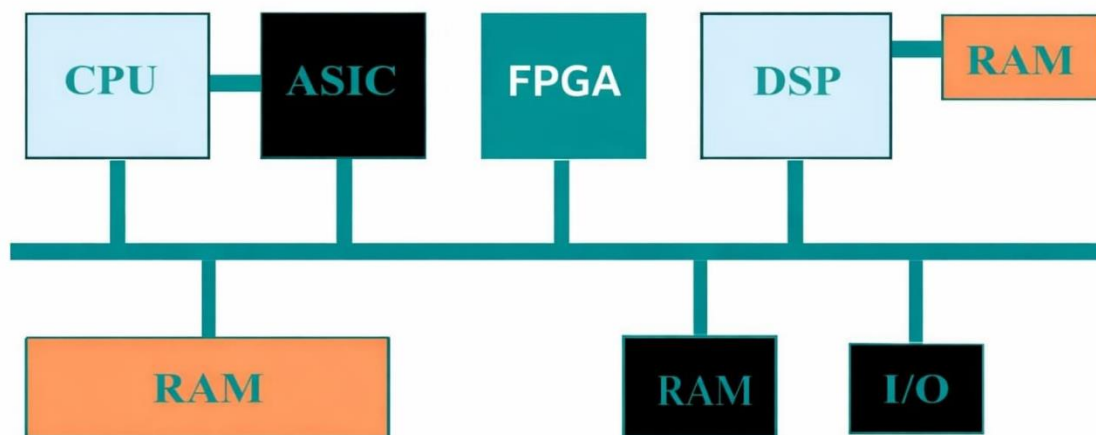


Fig 1.3 Heterogeneous architectures

1.8 Harsh environment

Many embedded systems do not operate in a controlled environment. Excessive heat is often a problem, especially in applications involving combustion (e.g., many transportation applications). Additional problems can be caused for embedded computing by a need for protection from vibration, shock, lightning, power supply fluctuations, water, corrosion, fire, and general physical abuse.

1.9 System safety and reliability

As embedded system complexity and computing power continue to grow, they are starting to control more and more of the safety aspects of the overall system. These safety measures may be in the form of software as well as hardware control. Mechanical safety backups are normally activated when the computer system loses control in order to safely shut down system operation. Software safety and reliability is a bigger issue. Software doesn't normally "break" in the sense of hardware. However, software may be so complex

that a set of unexpected circumstances can cause software failures leading to unsafe situations. Discussion of this topic is outside the scope of this book, but the challenges for embedded designers include designing reliable software and building cheap, available systems using unreliable components. The main challenge for embedded system designers is to obtain low-cost reliability with minimal redundancy.

1.10 Control of physical systems

One of the main reasons for embedding a computer is to interact with the environment. This is often done by monitoring and controlling external machinery. Embedded computers transform the analog signals from sensors into digital form for processing. Outputs must be transformed back to analog signal levels. When controlling physical equipment, large current loads may need to be switched in order to operate motors and other actuators. To meet these needs, embedded systems may need large computer circuit boards with many non-digital components. Embedded system designers must carefully balance system tradeoffs among analog components, power, mechanical, network, and digital hardware with corresponding software. Small and low weight Many embedded computers are physically located within some larger system. The form factor for the embedded system may be dictated by aesthetics. For example, the form factor for a missile may have to fit inside the nose of the missile. One of the challenges for embedded systems designers is to develop non-rectangular geometries for certain solutions. Weight can also be a critical constraint. Embedded automobile control systems, for example, must be light weight for fuel economy. Portable CD players must be light weight for portability purposes.

1.11 Cost sensitivity

Cost is an issue in most systems, but the sensitivity to cost changes can vary dramatically in embedded systems. This is mainly due to the effect of computer costs have on profitability and is more a function of the proportion of cost changes compared to the total system cost.

1.12 Power management

Embedded systems have strict constraints on power. Given the portability requirements of many embedded systems, the need to conserve power is important to

maintain battery life as long as possible. Minimization of heat production is another obvious concern for embedded system

1.13 DESIGN OF EMBEDDED SYSTEM:

Like every other system development design cycle embedded system too have a design cycle. The flow of the system will be like as given below steps.

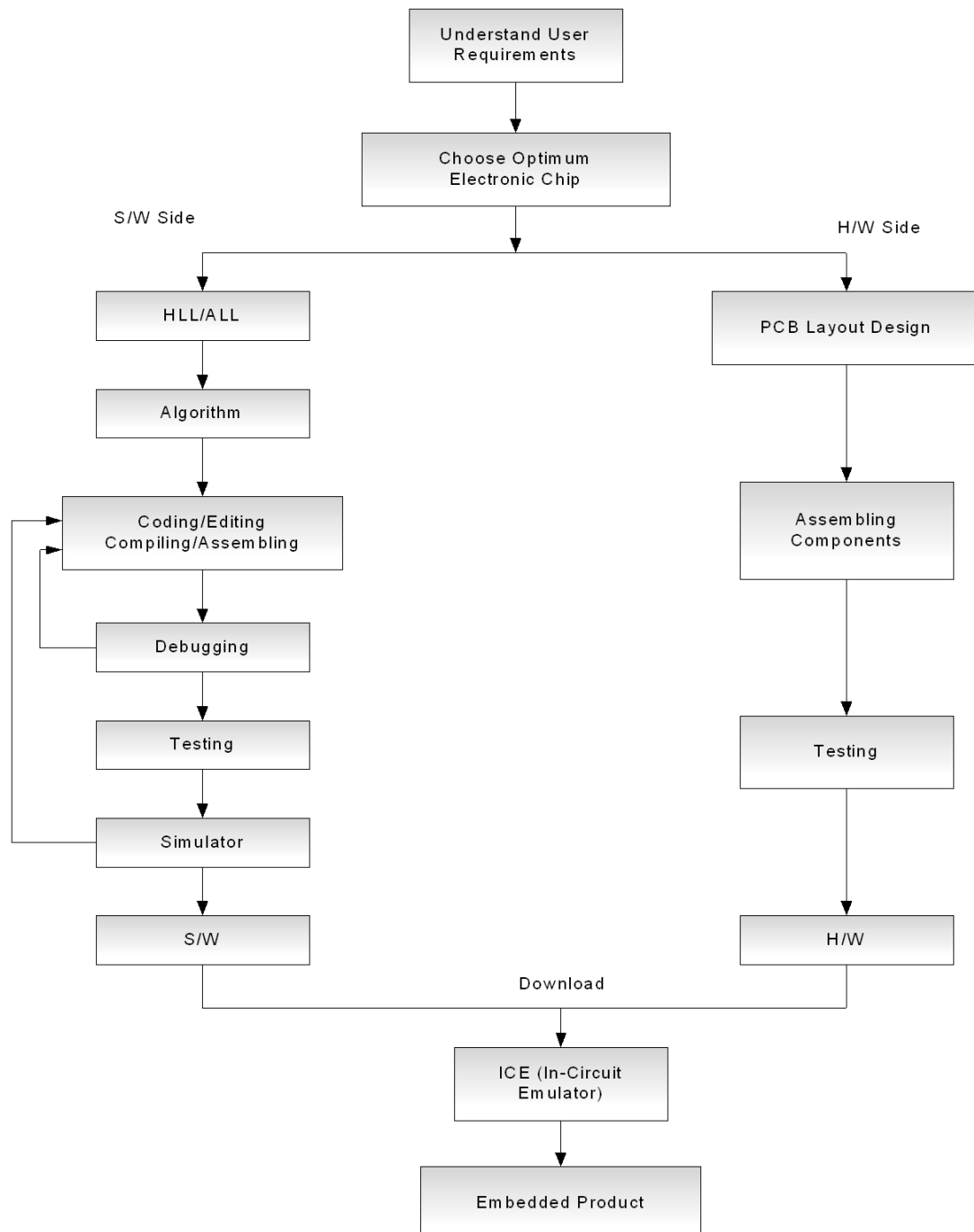


Fig 1.4 Design of embedded systems

From the initial state of the project to the final fabrication the design considerations will be taken like the software consideration and the hardware components, sensor, input and output. The electronics usually uses either a microprocessor or a microcontroller. Some large or old systems use general-purpose mainframe computers or minicomputers.

1.14 CLASSIFICATION:

- Real Time Systems.
- RTS is one which has to respond to events within a specified deadline.
- A right answer after the dead line is a wrong answer
- RTS classification
- Hard Real Time Systems
- Soft Real Time System

1.15 HARD REAL TIME SYSTEM:

- "Hard" real-time systems have very narrow response time.
- Example: Nuclear power system, Cardiac pacemaker.

1.16 SOFT REAL TIME SYSTEM:

- "Soft" real-time systems have reduced constrains on "lateness" but still must operate very quickly and repeatable.
- Example: Railway reservation system – takes a few extra seconds the data remains valid.

CHAPTER-2

LITERATURE SURVEY

Research on Lean Production Management Model of Xie Zhuang Coal Mine, Ruixi Zhang,2024

This paper introduces the lean production management mode of coal, and discusses the problems existing in the mining of XZ coal mine area iii based on the actual cases. Part of working procedures of fully mechanized mining face in the third mining area are analyzed, improvement plans are put forward, and the four principles of 5W1H questioning method and ECRS are applied for process improvement. By carrying out lean management of coal mine, reasonably matching personnel, reducing waste of time and resources, and reducing invalid labor time, the utilization rate of equipment can be improved to achieve the improvement purpose of "4M1E".

Research on Real-Time Monitoring System of Miners' Work and Health based on LPWAN, Jing Zhao Li,2023

The real-time monitoring system based on low-power wide area network (LPWAN) is built for the poor environment, as the high labor intensity, the heavy physical load and the poor health of the coal mine workers. The prediction method of miners' health status is proposed based on the long-term and short-term memory network (LSTM), and an intelligent sensing node with edge computing function is designed. The system can master the physical and working condition of miners in real time, and provide guarantee for the health of miners and the safety production of the coal mine.

LoRa WAN Based Smart Safety Helmet with Protection Mask for Miners, A G Vijayendra, 2022

In today's modern world, the demand for coal and the essential labor of coal miners persists, yet the inherent risks to their lives remain a pressing concern. An innovative solution emerges in the form of a smart helmet, an advanced iteration of the conventional headgear worn by miners. This intelligent helmet integrates vital features to enhance miners' safety and well-being. It enables real-time monitoring of critical parameters such as air quality, atmospheric pressure, radiation levels, and physiological indicators like

body temperature and overall health conditions. Additionally, the helmet incorporates a HEPA-filtered respiratory mask, effectively shielding miners from harmful airborne particles, including those responsible for the debilitating black lung disease. Leveraging LoRa wireless communication technology, the helmet securely transmits data to management desks, where it is further uploaded to cloud-based IoT networks, providing higher authorities with comprehensive insights into on-site conditions. The incorporation of a multi-layered filtration system within the mask underscores its efficacy in safeguarding miners' respiratory health amidst the challenging mining environment. Furthermore, a Heads-Up Display discreetly integrated into the helmet's visor offers miners real-time access to essential data and instructions without impeding their workflow, thus fostering a safer and more efficient work environment.

Development and Field Application of Ultrasonic Scanner for Coal Mine Gas Extraction Borehole Drilling Xingxing wang,2022

In the production process of coal mines, it is necessary to adopt the method of drilling to extract coal bed gas and reduce the risk of gas disaster in the process of coal bed mining. The geological conditions of coal mines in China are complex the coal industry proposes to adopt the method while drilling to measure the inner diameter of the borehole and the lithological distribution of the strata during the drilling process. Based on the basic principle of ultrasonic scanner while drilling, this paper proposes the use of a self-generating and self-receiving ultrasonic probe to emit ultrasonic waves and sample the echoes in the process of drilling. This paper develops the project's ultrasonic excitation module, ultrasonic echo acceptance module, data storage module, and upper computer communication module, and designs and processes a small-size borehole ultrasonic scanner instrument with an outer diameter of less than or equal to 89mm. The field test results show that the developed ultrasonic scanner while drilling is able to measure the ultrasonic echo data with clear envelopes, which meets the measurement demand of the borehole inner diameter in coal mine drill holes while drilling.

On relation between Work Study and safe-efficient coal mine, Yu-be Sun ,2015

Under the new situation of coal mine safety developing and coal production increasing rapidly, the author analyzed coal mines safety status, coal mine human error and underground production operations problems, clarified that the meaning of Work Study is in accordance with the connotation of safe-efficient coal mine construction in China, and

defined the connotation between safe-efficient Coal mine and work study, pointed out that using Work Study method can achieve the purpose of optimizing mine work procedures, methods of operation and operational actions, improving worker productivity, reducing the exposure time of risk in the work space and fatigue, and at last achieving safe-efficient coal mine production goals.

Fuzzy comprehensive evaluation of groundwater pollution in coal mining area **Run sheng Lv,2020**

The study objective is to propose a simple and efficient evaluation method. The groundwater pollutant sources were analyzed and the 20 groundwater samples & 34 chemical indexes were tested in Huainan mining area. Based on analytic testing results, the groundwater quality was evaluated with the method of fuzzy mathematical comprehensive evaluation, and pollution degree was proposed. The results indicate that the most groundwater has been polluted in research area, especially in coal mining industry center and utility area, which occupy 65% pollutions according to the evaluation criteria

Statistical Analysis of Coal Mine Accidents of China in 2018, Jun Chen ,2019

Coal mines constitute the basic industrial resource in China. China has a large number of coal mines, but insufficient safety measures cause frequent safety accidents, while the number of deaths per year in each mine has been ranked second, second only to traffic accidents. Firstly, in this paper, the 2018 mine accident data from the accident occurrence areas, the occurrence time, and the type of accidents were analyzed. Secondly, a detailed analysis was carried out regarding the large accidents from different provinces and cities. Finally, countermeasures and suggestions were proposed, based on the causes of the accidents. This data analysis could provide reference for future coal mine production safety, leading to the prevention and control emphasis of coal mine production safety.

Study on controlled by beds combination structure to deep of mining failure of coal seam floor, Wu Jiwen,2021

The large-scale exploitation of small coal mines has brought about a series of environmental problems over recent 30 years in China, such as the large-area failure and subsidence of ground, underground water resources having been dried up and serious air and water sources pollutions in mining districts. In order to protect the environment, it

is imperative to implement the reasonable exploitation of small coal mines. The issue expounds the specific measurements for ecological protection during Resource Integration among small coal mines. And it provided valuable ideas for the Resource Integrated Mine (RIM) to developing eco-economy, improving the ecological environment, and maintaining ecological safety under similar conditions.

The Design and Research of Coal Mine Safety Dynamic Evaluation System, Gao Shushan,2022

The rapid development of China coal mine safety management information plays an important role on improving mine safety standards, however, there exist some problems such as cannot be a comprehensive system of monitoring, evaluation and analysis of safety, forecasting of security trends, etc. Papers have researched based on this problem, carrying out research on an analysis and collection of security information, the acquisition of modeling, data processing, application of function evaluation and any other aspects. Then proposed and developed a system which has real-time forecasting and prediction function in the dynamic evaluation system for coal mine safety. Through this system, mine-related managers can real-time insight into production of underground safety and security trends, but also to navigate to a variety of insecurity and the workplace more quickly. The system designed and developed to improve mine safety and accident prevention, at present it has completed the initial development and online use already.

Risk Assessment of Smart Coal Mine Safety Mining Based on BIM, Xiao Wang,2022

Coal is an important basic energy source in our country, accounting for about 70% of the total energy, and its development prospects are very broad. Our country has abundant coal reserves, and 95% of coal mining is underground operations. Although major accidents have declined in the past two years, the safety production situation in our country's coal mines is still severe, with a large number of casualties every year. The BIM technology, which is currently in the research and promotion stage, provides an effective way to solve such problems. Promoting the application of BIM technology will be the trend of future development. However, the research on coal mining risk assessment is relatively lagging, and most of the original evaluation methods are not suitable for complex coal mine systems. Therefore, this article is mainly based on BIM-based smart coal mine safety mining RA (Risk Assessment) research. First of all, the analysis of the current situation of coal safety issues shows that the situation of CM (coal mine) safety production

in our country is still very severe. Secondly, analyzed the impact of BIM on coal mining RA. The results show that 41% of people believe that BIM technology can improve the accuracy of RA. 37% also pointed out that the application of BIM technology to coal mining RA can help save companies' capital cost.

2.1 Existing System

- Coal mining detection systems are usually based on infrasound, seismic, or radar signals.
- Within this study, we compared three different types of Coal mining detection systems: one avalanche radar, one infrasound array system consisting of four infrasound sensors, and a newly developed single sensor infrasound system.
- A special focus is given to the new single sensor system, which is a low cost, easy-to-install system, originally designed for the detection of debris flows and debris floods.
- Within this work, we analysed how this single sensor system could be adapted to also detect Coal mining.
- The radar system detected Coal mining accurately and reliably, but was limited to the particular Coal mining path towards which the radar beam was directed

CHAPTER 3

3.1 PROPOSED SYSTEM

- The proposed system is a low-power, long-range wearable health monitoring solution tailored for coal mine laborers. It consists of sensor-equipped wearable bands, a LoRa RF communication setup, and a remote receiver gateway connected to an IoT monitoring dashboard.
- The labors' wearable band is included in everyone's hands.
- To show the multiple parameters such as temperature, heart rate and SpO2 level, and coal mining area dust and humidity values.
- To transmit the LoRa RF based on sensor data.
- And get the receiver wireless medium
- If any emergency, they press the switch and get the receiver side messages.

3.2 Block Diagram

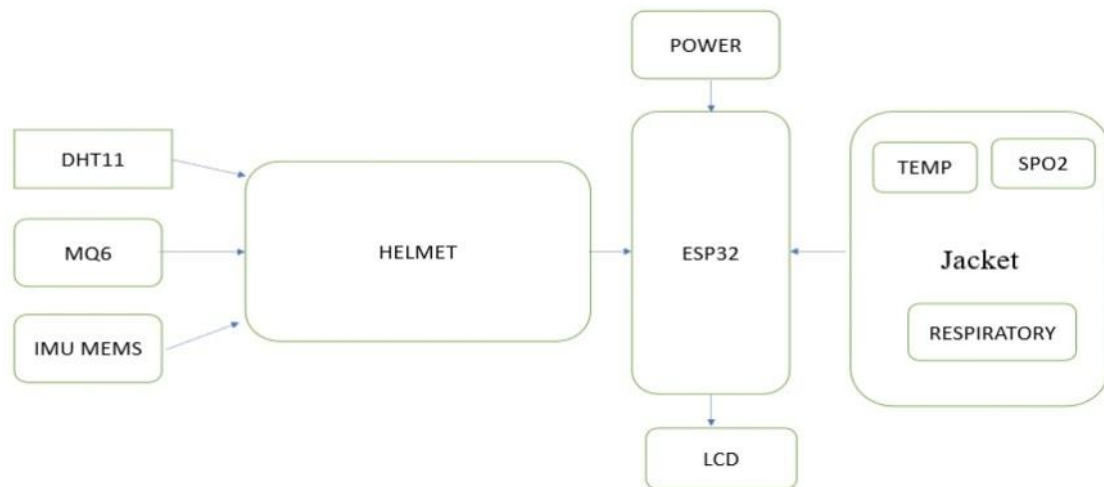


Fig 3.1 Block Diagram of Smart Safety Monitoring System Using ESP32

3.3 Hardware Components:

- ESP32
- DHT11
- MEMS
- Temperature Sensor
- Respiratory
- Heartbeat Sensor
- Helmet jacket

3.4 MODULES DESCRIPTION:

3.4.1 ESP32 MODULE:

Technology advanced, new project ideas and implementations came into play and one particular concept is the Internet of Things or IoT. It is a connected platform, where several “things” or devices are connected over internet for exchange of information.

In DIY community, the IOT projects are mainly focused on Home Automation and Smart Home applications but commercial and industrial IoT projects have far complex implementations like Machine Learning, Artificial Intelligence, Wireless Sensor Networks etc.

The important thing in this brief intro is whether it is a small DIY project by a hobbyist or a complex industrial project, any IoT project must have connectivity to Internet. This is where the likes of ESP8266 and ESP32 come into picture.

If you want to add Wi-Fi connectivity to your projects, then ESP8266 is a great option. But if you want build a complete system with Wi-Fi connectivity, Bluetooth connectivity, high resolution ADCs, DAC, Serial Connectivity and many other features, then ESP32 is the ultimate choice.

ESP32 is a low-cost System on Chip (SoC) Microcontroller from Express if Systems, the developers of the famous ESP8266 Soc It is a successor to ESP8266 SoC and comes in both single-core and dual-core variations of the Tensilica’s 32-bit Xtensa LX6 Microprocessor with integrated Wi-Fi and Bluetooth.

The good thing about ESP32, like ESP8266 is its integrated RF components like Power Amplifier, Low-Noise Receive Amplifier, Antenna Switch, Filters and RF Balun. This makes designing hardware around ESP32 very easy as you require very few external components.

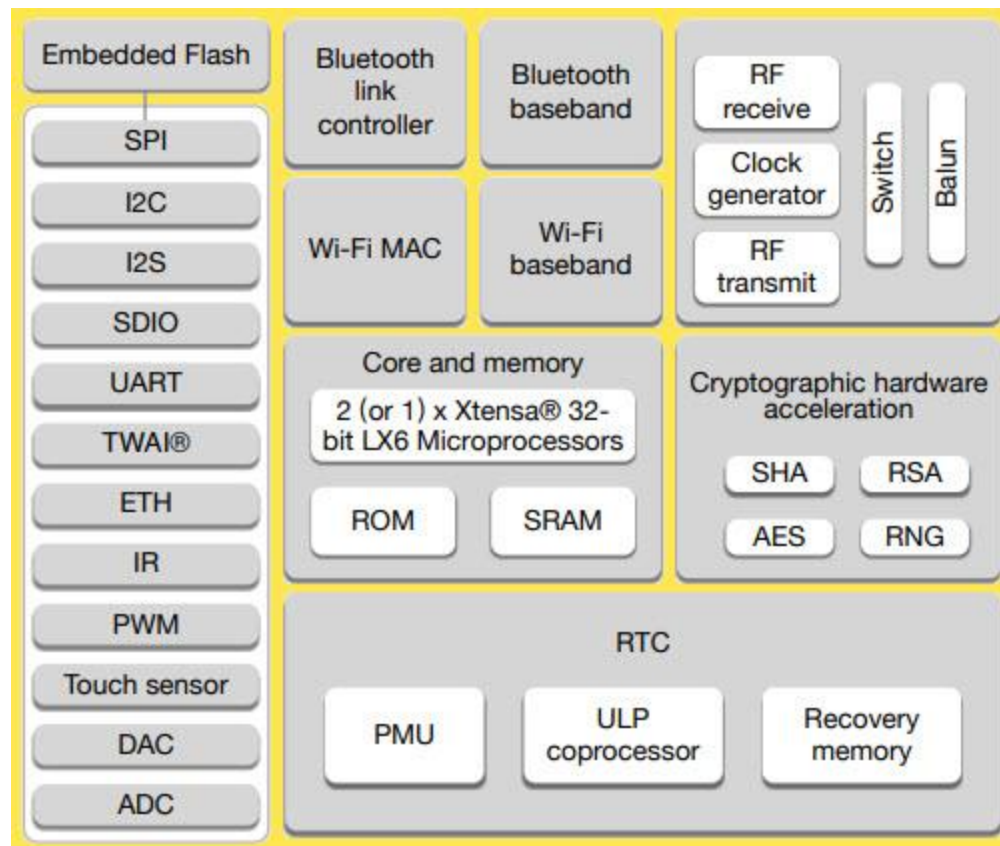


Fig 3.2 ESP32 Architecture

Another important thing to know about ESP32 is that it is manufactured using TSMC's ultra-low-power 40 nm technology. So, designing battery operated applications like wearables, audio equipment, baby monitors, smart watches, etc., using ESP32 should be very easy.

3.4.2 Specifications of ESP32

ESP32 has a lot more features than ESP8266 and it is difficult to include all the specifications in this Getting Started with ESP32 guide. So, I made a list of some of the important specifications of ESP32 here. But for complete set of specifications, I strongly suggest you to refer to the Datasheet.

- Single or Dual-Core 32-bit LX6 Microprocessor with clock frequency up to 240 MHz
- 520 KB of SRAM, 448 KB of ROM and 16 KB of RTC SRAM.
- Supports 802.11 b/g/n Wi-Fi connectivity with speeds up to 150 Mbps.
- Support for both Classic Bluetooth v4.2 and BLE specifications.
- 34 Programmable GPIOs.
- Up to 18 channels of 12-bit SAR ADC and 2 channels of 8-bit DAC
- Serial Connectivity include 4 x SPI, 2 x I²C, 2 x I²S, 3 x UART.
- Ethernet MAC for physical LAN Communication (requires external PHY).
- 1 Host controller for SD/SDIO/MMC and 1 Slave controller for SDIO/SPI.
- Motor PWM and up to 16-channels of LED PWM.
- Secure Boot and Flash Encryption.
- Cryptographic Hardware Acceleration for AES, Hash (SHA-2), RSA, ECC and RNG.

Different Ways to Program

A good hardware like ESP32 will be more user friendly if it can be programmed (writing code) in more than one way. And not surprisingly, the ESP32 supports multiple programming environments.

Some of the commonly used programming environments are:

- Arduino IDE
- Platform IO IDE (VS Code)
- LUA
- Micro Python
- Express if IDF (IoT Development Framework)
- JavaScript

As Arduino IDE is already a familiar environment, we will use the same to program ESP32 in our upcoming projects. But you can definitely try out others as well.

3.4.3 ESP32 Dev Kit – The ESP32 Development Board

Express if Systems released several modules based on ESP32 and one of the popular options is the ESP-WROOM-32 Module. It consists of ESP32 SoC, a 40 MHz crystal oscillator, 4 MB Flash IC and some passive components.



Fig 3.3 ESP32 Development Board

Layout

We will see what a typical ESP32 Development Board consists of by taking a look at the layout of one of the popular low-cost ESP Boards available in the market called the ESP32 Devkit Board.

The following image shows the layout of an ESP32 Development Board which I have.

IMPORTANT NOTE: There are many ESP32 Boards based on ESP-WROOM-32 Module available in the market. The layout, pinout and features vary from board to board.

The board which I have has 30 Pins (15 pins on each side). There are some boards with 36 Pins and some with slightly less Pins. So, double check the pins before making connections or even powering up the board.

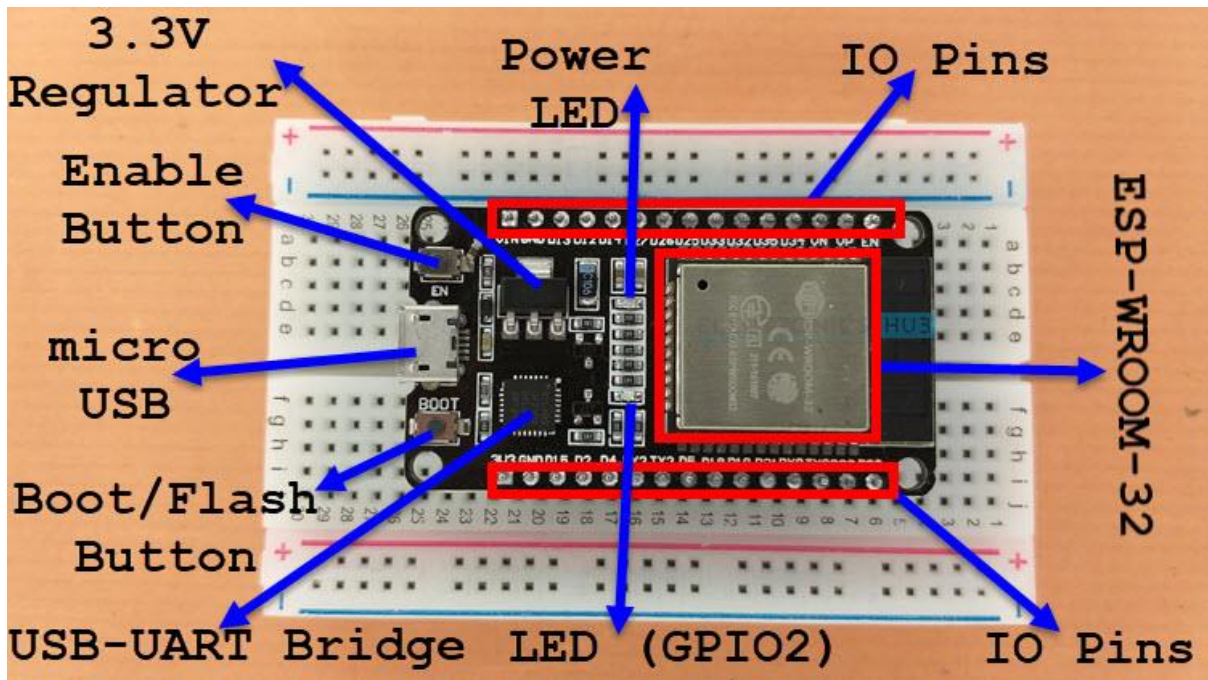


Fig 3.4 ESP32 Layout

As you can see from the image, the ESP32 Board consists of the following:

- ESP-WROOM-32 Module
- Two rows of IO Pins (with 15 pins on each side)
- CP2012 USB – UART Bridge IC
- micro–USB Connector (for power and programming)
- AMS1117 3.3V Regulator IC
- Enable Button (for Reset)
- Boot Button (for flashing)
- Power LED (Red)
- User LED (Blue – connected to GPIO2)

- Some passive components

An interesting point about the USB-to-UART IC is that its DTR and RTS pins are used to automatically set the ESP32 in to programming mode (whenever required) and also reset the board after programming.

3.5 Pinout of ESP32 Board

I will make a separate dedicated tutorial on ESP32 Pinout. But for the time being, take a look the pinout diagram of the ESP32 Development Board.

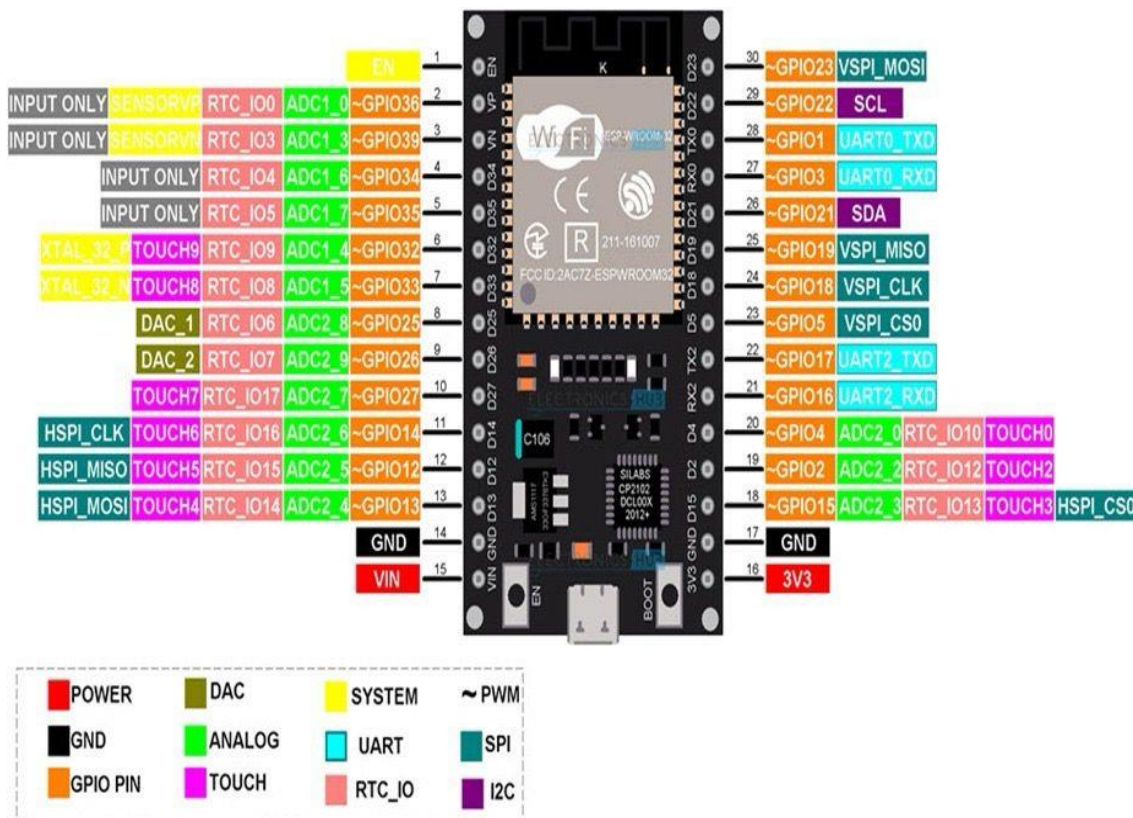


Fig 3.5 Pin Diagram of ESP32

3.6 TEMPERATURE SENSOR:

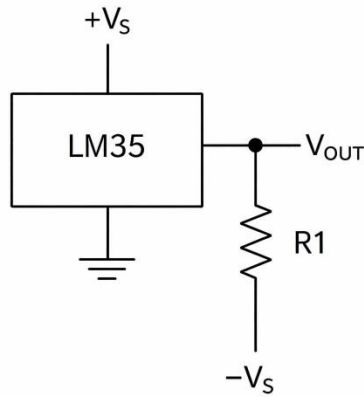
The LM35 series are precision integrated-circuit temperature devices with an output voltage linearly proportional to the Centigrade temperature. The LM35 device has an advantage over linear temperature sensors calibrated in Kelvin, as the user is not required to subtract a large constant voltage from the output to obtain convenient Centigrade scaling. The LM35 device does not require any external calibration or trimming to provide typical accuracies of $\pm 1/4^{\circ}\text{C}$ at room temperature and $\pm 3/4^{\circ}\text{C}$ over a full -55°C to 150°C

temperature range. Lower cost is assured by trimming and calibration at the wafer level. The low-output impedance, linear output, and precise inherent calibration of the LM35 device makes interfacing to readout or control circuitry especially easy. The device is used with single power supplies, or with plus and minus supplies. As the LM35 device draws only 60 μA from the supply, it has very low self-heating of less than 0.1°C in still air. The LM35 device is rated to operate over a -55°C to 150°C temperature range, while the LM35C device is rated for a -40°C to 110°C range (-10° with improved accuracy). The LM35-series devices are available packaged in hermetic TO transistor packages, while the LM35C, LM35CA, and LM35D devices are available in the plastic TO-92 transistor package. The LM35D device is available in an 8-lead surface-mount small-outline package and a plastic TO-220 package.

Table 1: LM35 Package Types and Physical Dimensions

PART NUMBER	PACKAGE	BODY SIZE(NOM)
LM35	TO-CAN (3)	4.699 mm $\times$ 4.699 mm
	TO-92(3)	4.30 mm $\times$ 4.30 mm
	SOIC (8)	4.90 mm $\times$ 3.91 mm
	TO-220(3)	14.986 mm $\times$ 10.16 mm

Full-Range Centigrade Temperature Sensor



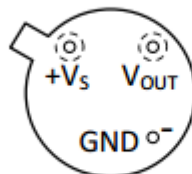
Choose $R_1 = -V_s / 50 \mu\text{A}$

$V_{OUT} = 1500 \text{ mV}$ at 150°C

$V_{OUT} = 250 \text{ mV}$ at 25°C

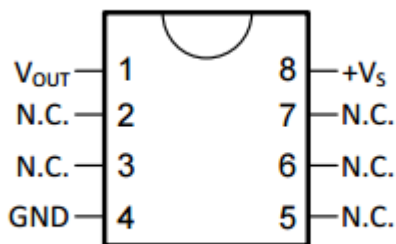
$V_{OUT} = -550 \text{ mV}$ at -55°C

NDV Package
3-Pin TO-CAN
(Bottom View)



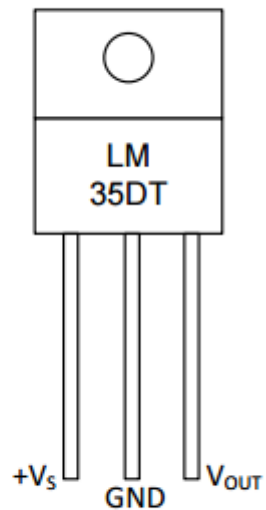
Case is connected to negative pin (GND)

D Package
8-PIN SOIC
(Top View)



N.C. = No connection

NEB Package
3-Pin TO-220
(Top View)



Tab is connected to the negative pin (GND).

NOTE: The LM35DT pinout is different than the discontinued LM35DP

Fig 3.6 Full-Range Centigrade Temperature Sensor

The LM35 is easily applied in the same way as other integrated-circuit temperature sensors. Glue or cement the device to a surface and the temperature should be within about 0.01°C of the surface temperature. The 0.01°C proximity presumes that the ambient air temperature is almost the same as the surface temperature. If the air temperature were much higher or lower than the surface temperature, the actual temperature of the LM35 die would be at an intermediate temperature between the surface temperature and the air temperature; this is especially true for the TO-92 plastic package. The copper leads in the TO-92 package are the principal thermal path to carry heat into the device, so its temperature might be closer to the air temperature than to the surface temperature. Ensure that the wiring leaving the LM35 device is held at the same temperature as the surface of interest to minimize the temperature problem. The easiest fix is to cover up these wires with a bead of epoxy. The epoxy bead will ensure that the leads and wires are all at the same temperature as the surface, and that the temperature of the LM35 die is not affected by the air temperature. The TO-46 metal package can also be soldered to a metal surface or pipe without damage. Of course, in that case the V-terminal of the circuit will be grounded to that metal. Alternatively, mount the LM35 inside a sealed end metal tube, and then dip into a bath or screw into a threaded hole in a tank. As with any IC, the LM35 device and accompanying wiring and circuits must be kept insulated and dry, to avoid leakage and corrosion. This is especially true if the circuit may operate at cold temperatures where condensation can occur. Printed-circuit coatings and varnishes such as a conformal coating and epoxy paints or dips are often used to ensure that moisture cannot corrode the LM35 device or its connections. These devices are sometimes soldered to a small light-weight heat fin to decrease the thermal time constant and speed up the response in slowly-moving air. On the other hand, a small thermal mass may be added to the sensor, to give the steadiest reading deUARTte small deviations in the air temperature

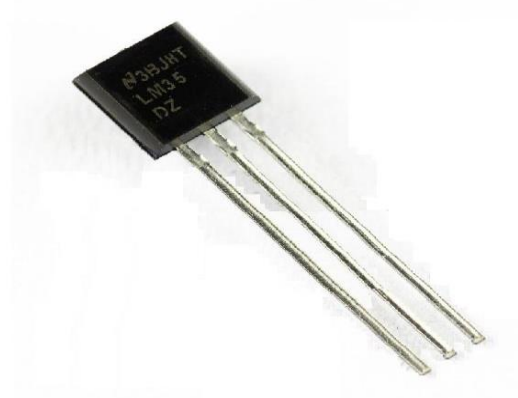


Fig 3.7 Temperature Sensor

3.7 HEARTBEAT SENSOR

A person's heartbeat is the sound of the valves in his/her's heart contracting or expanding as they force blood from one region to another. The number of times the heart beats per minute (BPM), is the heart beat rate and the beat of the heart that can be felt in any artery that lies close to the skin is the pulse.

3.7.1 Two Ways to Measure a Heartbeat

Manual Way: Heart beat can be checked manually by checking one's pulses at two locations- wrist (the radial pulse) and the neck (carotid pulse). The procedure is to place the two fingers (index and middle finger) on the wrist (or neck below the windpipe) and count the number of pulses for 30 seconds and then multiplying that number by 2 to get the heart beat rate. However pressure should be applied minimum and also fingers should be moved up and down till the pulse is felt. Using a sensor Heart Beat can be measured based on optical power variation as light is scattered or absorbed during its path through the blood as the heart beat changes.

3.7.2 Principle of Heartbeat Sensor

The heartbeat sensor is based on the principle of photo plethysmography. It measures the change in volume of blood through any organ of the body which causes a change in the light intensity through that organ (a vascular region). In case of applications where heart pulse rate is to be monitored, the timing of the pulses is more important. The flow of blood volume is decided by the rate of heart pulses and since light is absorbed by

blood, the signal pulses are equivalent to the heart beat pulses. There are two types of photoplethysmography

Transmission: Light emitted from the light emitting device is transmitted through any vascular region of the body like earlobe and received by the detector.

Reflection: Light emitted from the light emitting device is reflected by the regions.

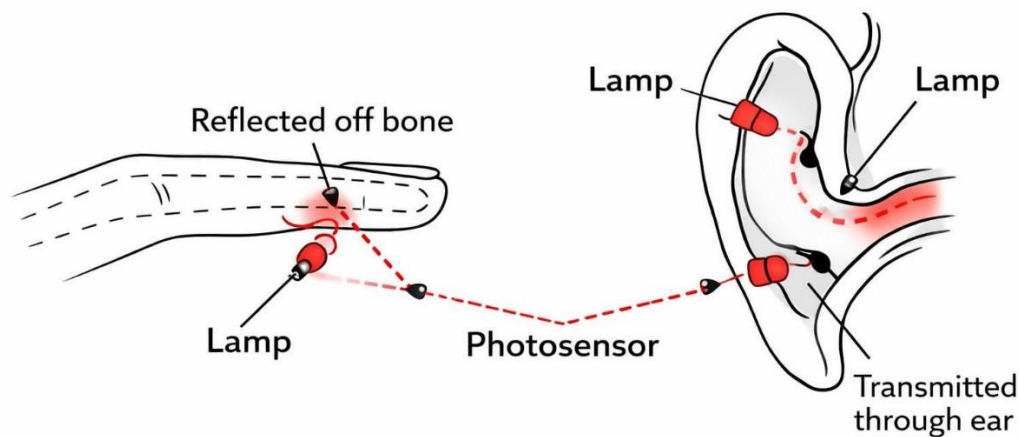


Fig 3.8 Principle of Heartbeat Sensor

3.7.3 Working of a Heartbeat Sensor

The basic heartbeat sensor consists of a light emitting diode and a detector like a light detecting resistor or a photodiode. The heart beat pulses cause a variation in the flow of blood to different regions of the body. When a tissue is illuminated with the light source, i.e. light emitted by the led, it either reflects a finger tissue or transmits the light (earlobe). Some of the light is absorbed by the blood and the transmitted or the reflected light is received by the light detector. The amount of light absorbed depends on the blood volume in that tissue. The detector output is in form of electrical signal and is proportional to the heart beat rate. This signal is actually a DC signal relating to the tissues and the blood volume and the AC component synchronous with the heart beat and caused by pulsatile changes in arterial blood volume is superimposed on the DC signal. The major requirement is to isolate that AC component as it is of prime importance.

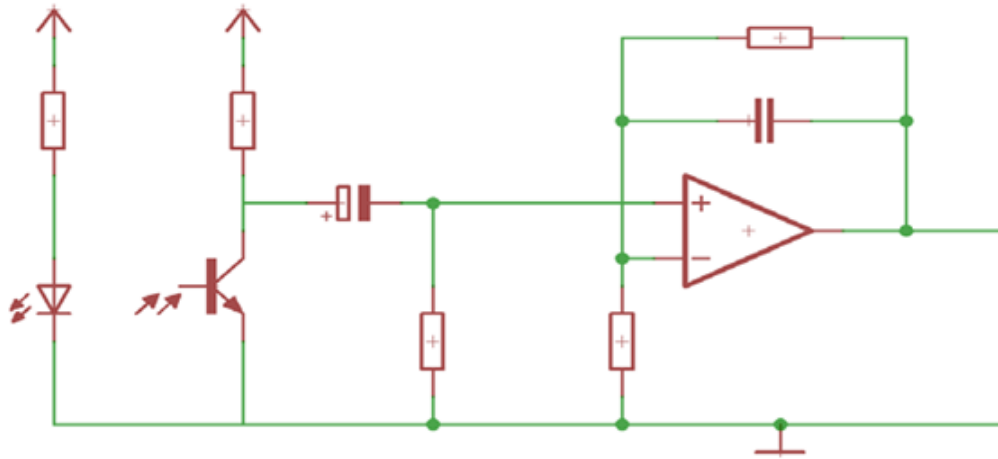


Fig 3.9 Circuit of Heartbeat Sensor

To achieve the task of getting the AC signal, the output from the detector is first filtered using a 2 stage HP-LP circuit and is then converted to digital pulses using a comparator circuit or using simple ADC. The digital pulses are given to a microcontroller calculating the heart beat rate,

$$\text{BPM (Beats per minute)} = 60 * f$$

Where f is the pulse frequency

3.7.4 Practical Heartbeat Sensor

Practical heartbeat Sensor examples are Heart Rate Sensor (Product No PC-3147). It consists of an infrared led and a ldr embedded onto a cliplike structure. The clip is attached to the organ (earlobe or the finger) with the detector part on the flesh.

Another example is TCRT1000, having 4 pins-

Pin1: To give supply voltage to the LED

Pin2 and 3 are grounded. Pin 4 is the output. Pin 1 is also the enable pin and pulling it high turns the LED on and the sensor starts working. It is embedded on a wearable device which can be worn on the wrist and the output can be sent wirelessly (through Bluetooth) to the computer for processing.



Fig 3.10 Heartbeat Sensor

3.7.5 Application Developing your own Heartbeat Sensor System

A basic Heartbeat Sensor system can also be built using basic components like comparator IC LM358 and a Microcontroller as given below

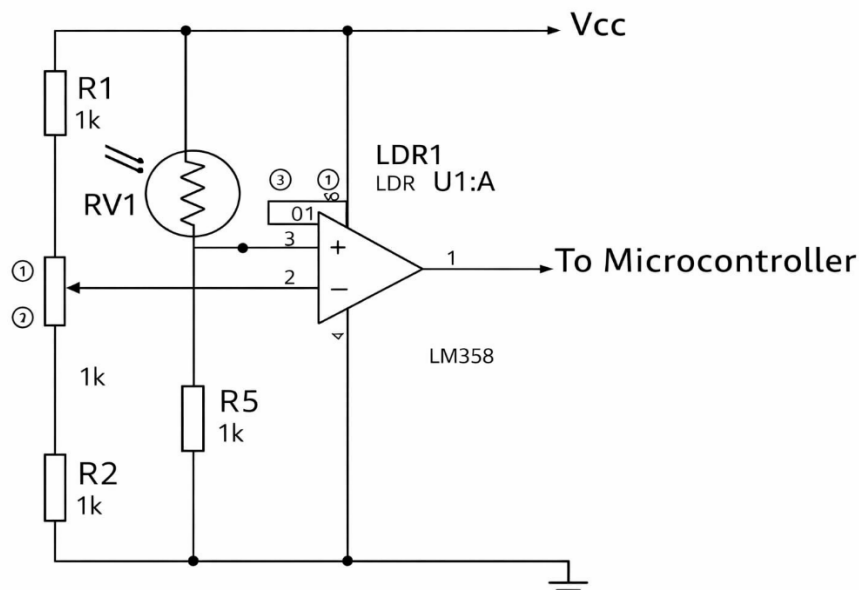


Fig 3.11 LDR-Based Heartbeat Sensor Circuit Diagram

As described above regarding the principle of heart beat sensor, when the finger tissue or the earlobe tissue is illuminated using a light source, the light is transmitted after getting modulated i.e. a part getting absorbed by the blood and the rest being transmitted. This modulated light is received by the light detector. Here a Light Dependent Resistor (LDR)

is used as a light detector. It works on the principle that when light falls on the resistor, its resistance changes. As the light intensity increases, the resistance decreases. Thus, the voltage drops across the resistor decreases. Here a comparator is used which compares the output voltage from the LDR to that of the threshold voltage. The threshold voltage is the voltage drop across the LDR when the light with fixed intensity, from the light source falls directly on it. The inverting terminal of the comparator LM358 is connected to the potential divider arrangement which is set to the threshold voltage and the non-inverting terminal is connected to the LDR. When a human tissue is illuminated using the light source, the intensity of the light reduces. As this reduced light intensity falls on the LDR, the resistance increases and as a result the voltage drop increases. When the voltage drops across the LDR or the non-inverting input exceeds that of the inverting input, a logic high signal is developed at the output of the comparator and in case voltage drop being lesser a logic low output is developed. Thus, the output is a series of pulses.

3.8 RESPIRATORY SENSOR

3.8.1 GENERAL DESCRIPTION

The Respiration Sensor is used to monitor abdominal or thoracical breathing, in biofeedback applications such as stress management and relaxation training. Besides measuring breathing frequency, this sensor also gives you an indication of the relative depth of breathing. The Respiration Sensor for Nexus can be worn over clothing, although for best results we advise that there only be 1 or 2 layers of clothing between the sensor and the skin. The Respiration Sensor is usually placed in the abdominal area, with the central part of the sensor just above the navel. The sensor should be placed tight enough to prevent loss of tension.

3.8.2 PRODUCT DESCRIPTION

First Sensor develops and manufactures highly reliable sensors and customized sensor systems as a strategic partner to medical product manufacturers in the area of breathing and respiration. The first step in this process is breathing in air, or inhaling. The taking in of air rich in oxygen into the body is called inhalation and giving out of air rich in carbon dioxide from the body is called exhalation. The second step is gas exchange in the lungs where oxygen is diffused into the blood and the carbon dioxide diffuses out of the blood. The third process is cellular respiration, which produces the chemical energy

that the cells in the body need, and carbon dioxide. Finally, the carbon dioxide from cellular respiration is breathed out of body from the lungs.



Fig 3.12 Respiratory sensor

3.8.3 FEATURES

- Input voltage: 5v
- Output voltage: 5v
- Output: Analog
- Range: 30% – 65%
- Size (Approx.): 132cm (52" Long)

3.8.4 APPLICATIONS

- Medical purpose
- Environmental Control System
- Emergency response System

3.9 Micro-electromechanical systems (MEMS)

Micro-electromechanical systems (MEMS) is a technology that combines computers with tiny mechanical devices such as sensors, valves, gears, mirrors, and actuators embedded in semiconductor chips. MEMS or what he calls analogy computing will be "the foundational technology of the next decade." MEMS is also sometimes called smart matter.

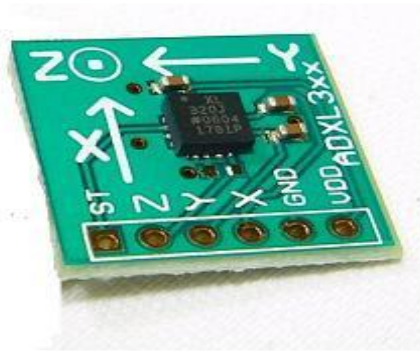


Fig 3.13 MEMS

MEMS are already used as accelerometers in automobile air-bags. They've replaced a less reliable device at lower cost and show promise of being able to inflate a bag not only on the basis of sensed deceleration but also on the basis of the size of the person they are protecting. Basically, a MEMS device contains micro-circuitry on a tiny silicon chip into which some mechanical device such as a mirror or a sensor has been manufactured. Potentially, such chips can be built in large quantities at low cost, making them cost-effective for many uses. Global position system sensors that can be included with courier parcels for constant tracking and that can also sense parcel treatment en route

- Sensors built into the fabric of an airplane wing so that it can sense and react to air flow by changing the wing surface resistance; effectively creating a myriad of tiny wing flaps

- Optical switching devices that can switch light signals over different paths at 20-nanosecond switching speeds
- Sensor-driven heating and cooling systems that dramatically improve energy savings
- Building supports with imbedded sensors that can alter the flexibility properties of a material based on atmospheric stress sensing

3.10 Applications:

- Vibrations measurement and Tilt inclination (to adjust rotation speed, circuit breaker security, fault prediction),
- Professional and domestic Alarms
- Maintenance management and Asset Tracking for shocks monitoring
- Alarm and Anti-Tamper function

3.11 Humidity sensor

Measurement and control of temperature and relative humidity find applications in numerous areas. These days devices are available which have both temperature and humidity sensors with signal conditioning, ADC, calibration and communication interface all built inside them. The use of such smart sensors greatly simplifies the design and reduces the overall cost. We discussed in past about. These sensors are capable of measuring both temperature and relative humidity and provide fully calibrated digital outputs. While SHT1x/SHT7x are very accurate sensors, they are still expensive for hobbyists use. This article discusses the DHT11 sensor which also provides calibrated digital outputs for temperature and humidity but is relatively lot cheaper than the Sensation sensors. The DHT11 sensor uses a proprietary 1-wire protocol which we will be exploring here and implementing with the PIC16F628A microcontroller that will receive the temperature and humidity values from the sensor and display them on a 16×2-character LCD.

3.11.1 About DHT11 sensor

The DHT11 sensor comes in a single row 4-pin package and operates from 3.5 to 5.5V power supply. It can measure temperature from 0-50 °C with an accuracy of $\pm 2^{\circ}\text{C}$ and relative humidity ranging from 20-95% with an accuracy of $\pm 5\%$. The sensor provides fully calibrated digital outputs for the two measurements. It has got its own proprietary 1-wire protocol, and therefore, the communication between the sensor and a microcontroller is not possible through a direct interface with any of its peripherals. The protocol must be implemented in the firmware of the MCU with precise timing required by the sensor. The following timing diagrams describe the data transfer protocol between a MCU and the DHT11 sensor. The MCU initiates data transmission by issuing a “Start” signal. The MCU pin must be configured as output for this purpose. The MCU first pulls the data line low for at least 18 mms and then pulls it high for next 20-40? s before it releases it.

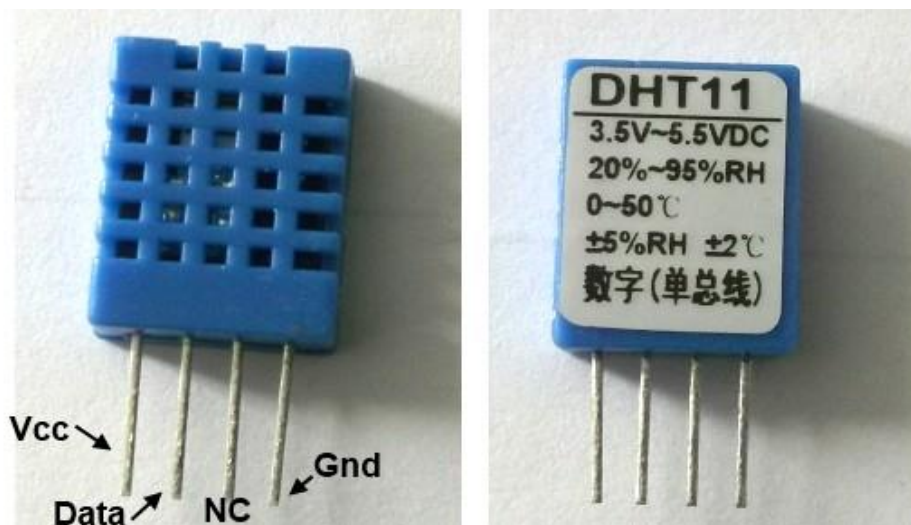


Fig 3.14 DHT11 sensor comes in a single row 4-pin package

Next, the sensor responds to the MCU “Start Signal by pulling the line low for 80? s followed by a logic high signal that also lasts for 80? s. Remember that the MCU pin must be configured to input after finishing the “Start “signal. Once detecting the response signal from the sensor, the MCU should be ready to receive data from the sensor. The sensor then sends 40 bits (5 bytes) of data continuously in the data line. Note that while transmitting bytes, the sensor sends the most significant bit first.

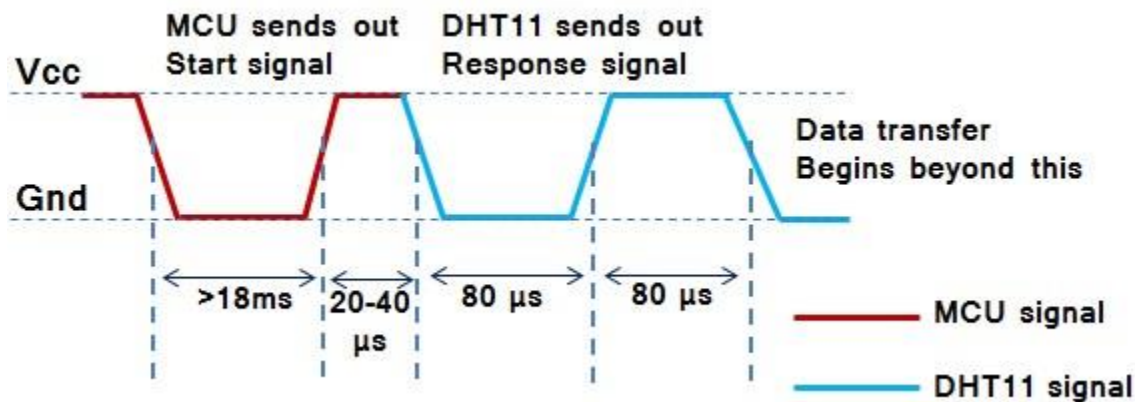


Fig 3.15 "Start" and "Response" signals

The 40-bit data from the sensor have the following structure.

Data (40-bit) = Integer Byte of RH + Decimal Byte of RH + Integer Byte of Temp. + Decimal Byte of Temp. + Checksum Byte

For DHT11 sensor, the decimal bytes of temperature and humidity measurements are always zero. Therefore, the first and third bytes of received data actually give the numeric values of the measured relative humidity (%) and temperature (°C). The last byte is the checksum byte which is used to make sure that the data transfer has happened without any error. If all the five bytes are transferred successfully then the checksum byte must be equal to the last 8 bits of the sum of the first four bytes, i.e.,

Checksum = Last 8 bits of (Integer Byte of RH + Decimal Byte of RH + Integer Byte of Temp. + Decimal Byte of Temp.)

Now let's talk about the most important thing, which is signaling for transmitting "0" and "1". In order to send a bit of data, the sensor first pulls the line low for 50 μs. Then it raises the line too high for 26-28 μs if it has to send "0", or for 70 μs if the bit to be transmitted is "1". So, it is the width of the positive pulse that carries information about 1 and 0.

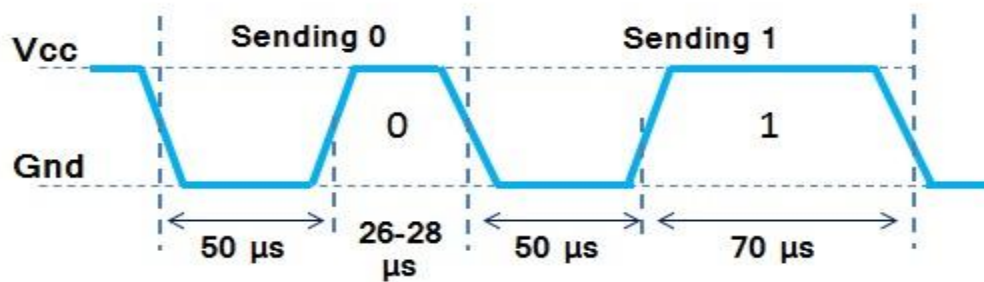


Fig 3.16 Timing difference for transmitting "1s" and "0s"

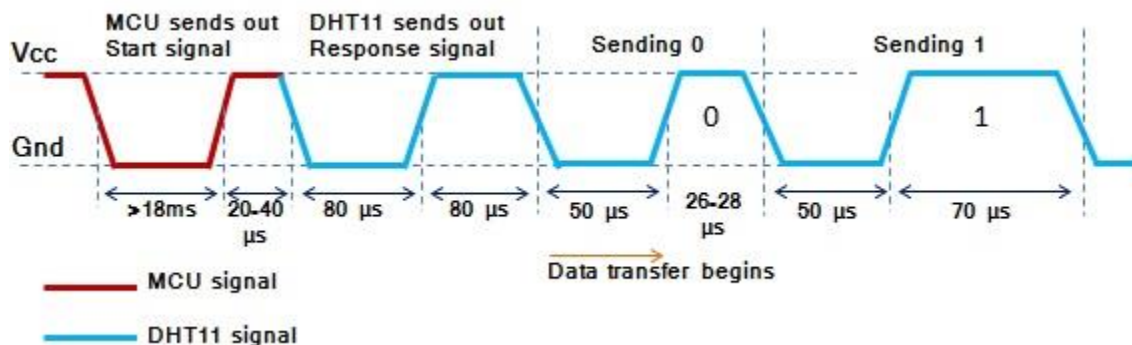


Fig 3.17 Start, Response and Data signals in sequence

At the end of the last transmitted bit, the sensor pulls the data line low for 50 μs and then releases it. The DHT11 sensor requires an external pull-up resistor to be connected between its Vcc and the data line so that under idle condition, the data line is always pulled high. After finishing the data transmission and releasing the data line, the DHT11 sensor goes to the low-power consumption mode until a new "Start" signal arrives from the MCU.

3.12 EMBEDDED C

This course introduces the C programming language and specifically addresses the issue of embedded programming. It is assumed that you have worked with some other high-level language before, such as Python, BASIC, FORTRAN or Pascal. Due to the complexities of embedded systems, we begin with a typical desktop system and examine the structure of the language along with basic examples. Once we have a decent grounding in syntax, structure, and the development cycle, we switch over to an

embedded system, namely an Arduino based development system. This course is designed so that you can do considerable work at home with minimal cost, if you choose (entirely optional, but programming these little beasts can be addicting so before warned). Along with this course text you will need an Arduino Uno board and a USB host cable. A small “wall wart” power adapter for it may also be useful. There’s a lot of free C programming info on the ‘net but if you prefer print books and want more detail, you may also wish to purchase one of the many C programming texts available. Two good titles are Kochan’s book Programming in C and the one by Deitel & Deitel C-How to Program. Whichever book you choose, make sure that its focus is C, not C++. You will also need a desktop C compiler. Just about any will do, including Visual C/C++, Borland, Code Warrior, or even GCC. A couple of decent freeware compilers available on the ‘net include Pelles C and Miracle C.

- It is a modern structured language that has been standardized (ANSI).
- It is modular, allowing reuse of code.
- It is widely supported, allowing source code to be used for several different platforms by just recompiling for the new target.
- Its popularity means that several third-party add-ons (libraries and modules) are available to “stretch” the language.
- It has type checking which helps catch errors.
- It is very powerful, allowing you to get “close to the metal”.
- Generally, it creates very efficient code (small space and fast execution).

Consider a typical embedded application such as a programmable or “intelligent” thermostat. Unlike a normal electro-mechanical thermostat, these devices allow the home owner to automatically change temperature at different times of the day in order to save energy. After all, why have the heat or air conditioner running when no one’s home. Certainly, these devices do not come with a monitor or keyboard. In their place may be a small LCD display with a few fixed messages and a two-digit numeric display for the temperature. For input there may be as few as two or three buttons for programming (set item plus up and down). By comparison, a microwave oven will probably have a complete numeric keypad with a series of special function buttons along with multiple seven-

segment displays, or possibly several alpha-numeric displays for short messages. In any case, these devices are far different from the standard desktop computer. Consequently, a programmer's approach to input and output processing will be much different in the embedded case. To start with, it is unlikely that there will be `print ()` and `scan ()` style functions. They are largely worthless in this world. What use would `print ()` be if all you have for output hardware is a bunch of LEDs? For input, you often need to read the state of switches, pushbuttons, and possibly some form of level control such as a rotary knob. For output, you often need to simply light an LED or set a value on a seven-segment display. For "fixed" style messages, these also need only a single signal to turn them on or off, such as an LED. In more advanced applications, a multi-line alphanumeric display may be available so setting individual letters is a possibility. In almost all cases these chores are handled by setting or clearing bits on specific output or inputs ports on the microcontroller. Some ports may be set up as a byte or word. Further, some ports may be bi-directional, meaning that they can behave as either input or output depending on some other register setting. Ports are little more than pins on the microcontroller that are hooked up to external circuitry. Thus, if a port is connected to an LED circuit, setting the output of the port HIGH could light the LED while setting the port LOW could turn off the LED. The obvious question then is "How do you read from or write to a port?" In many cases ports will be memory mapped. That is, a specific address in the memory map is allocated to a given port. You read and write from/to it just like any other variable. Further, development systems sometimes disguise these addresses as pre-defined global variables. They might also include a library of specific routines to utilize the ports. Thus, setting a certain port (let's call it the "A" port) to a high might be as simple as `PORT_A = 1;` or `set porta (1);` Reading from a port might be something like `a = PORT_A;` or `a = get porta ();` Consequently, embedded code is often all about reading and writing to/from ports and then branching to the requested chores.

Embedded Controllers 75

There are some tricks to this. For example, how do you know if a key has been pressed? Calling `get porta ()` tells you the state of the switch connected to port A at the instant you call it. There is no "history" here if this is a simple momentary pushbutton instead of a toggle switch. In a case like this you might "poll" the port waiting for something to happen. This involves using a simple loop and repeatedly reading the port state in the loop. The code breaks

out when the state changes: `while get port A ();` This will keep looping until `get port A ()` returns 0. Of course, if you need to monitor several ports as is typical, you'll need to read a value from each for the test. This form of monitoring while waiting for something to happen is called an event loop. It may not be evident, but your house and car are probably filled with devices running event loops, just waiting for you to do something! These loops execute fairly fast so a time lag between your push and the resulting action is not noticed. On the output end, a port normally stays at the value you set it, so there is no need for a loop to "keep it set". For more complicated displays such as a seven segment or alpha-numeric device, you may need to create a table of values indicating bit patterns for each numeral or letter to be displayed. These patterns, or words, would then be sent out various ports that are in turn connected to the displays. For variable input devices such as a volume control, the external device (knob, slider, sensor, etc.) will be connected to an analog to digital converter or ADC. This might be a separate circuit or the controller may have these built-in. The converter turns the analog voltage into a numeric value that you can read from a port. For example, a volume control may be just a potentiometer connected to a fixed voltage. As the knob is moved, the voltage at the wiper arm will change. The A/D converter may encode this into a single byte. A byte ranges from 0 to 255 in value. Thus, if the volume is at maximum, the port will read 255. If the volume is at halfway the port will read about 127, and finally 0 when the volume is off

3.13 Memory and Hardware

The real "kicker" is that you can't do native development with embedded code. In other words, you can't program the microcontroller just using the microcontroller the way you can create desktop applications using a desktop computer. Instead, you need to have a host and a target. The host is the computer you use for development such as a normal desktop unit while the target is the thing you're developing for the embedded application. The compiler that you use is technically referred to as a cross-compiler because it creates machine code for a processor other than the one the host uses. For example, your PC might use a Pentium processor, but the cross-compiler that runs on it creates machine code for a specific Atmel AVR microcontroller. To test your code, you need to either

simulate the target on the host, or you can download the compiled code to the target and test it there. This is an extra, but unavoidable, step.

Free Memory

In a given computer, memory is used by the operating system as well as by any running applications. Any memory left over is considered to be part of the “free memory pool”. This pool is not necessarily contiguous. It may be broken up into several different sized chunks. It all depends on the applications being run and how the operating system deals with them. The total amount of free memory and the locations of the various chunks will change over time. C offers ways of “asking” the operating system for a block of memory from the free pool. If the operating system can grant your request, you will have access to the memory and can use it as you see fit. When you are through using the memory, you tell the operating system that you are done with it so that it can reuse it elsewhere. Sounds simple,

Another important aspect is memory. Note that the AVR contains several different kinds of memory including flash program memory, static data memory or SRAM (Static Random Access Memory) and EEPROM (Electrically Erasable Programmable Read-Only Memory). Typical embedded applications run a single program repeatedly. This program must run from the moment the device is turned on until it is turned off. In many applications, this program is never updated (a microwave oven, perhaps). In other applications, it may be updated but only by qualified technicians, not by the consumer (for example, an automotive engine management system). Consequently, the program memory needs to be non-volatile, that is, it must be able to survive without power. Historically, this was done via ROM (Read-Only Memory) or PROM (Programmable Read-Only Memory). In both of these, individual bits can be set or cleared through the use of fuse/anti-fuse links. The difference is that ROMs are programmed at the time of manufacture while PROMs are programmed after manufacture. Both are permanent and cannot be reprogrammed. ROM is less expensive for large production runs but more expensive in small quantities. EEPROM has the advantage of being erasable and thus is a form of non-volatile read/write storage. Flash RAM is similar but is much less expensive. It has the downside that it must be programmed in blocks whereas EEPROM is byte-

programmable. Therefore, Flash RAM (also called NVRAM) is ideal as a storage medium for the main program and EEPROM is useful as a medium for storing data that must survive a power cycle. A possible example use of EEPROM would involve saving user preference settings for a digital camera. When power is turned off and back on, the user expects the device to be “as they left it”, not reverting back to a default state. Typically, special instructions or procedures are needed to write to or read from EEPROM. In contrast to Flash RAM and EEPROM, SRAM is volatile, just like the common DRAM (Dynamic RAM) variants found in personal computers. Static RAM typically is made of six transistors arranged as a flip-flop for each bit of storage. DRAM, by comparison, typically consists of a single transistor-capacitor combination per bit. SRAM is less dense and more expensive per bit than DRAM but it is very fast and does not need to be refreshed (the charge in a DRAM cell will leak over time requiring the cell to be refreshed at regular intervals). As a result, SRAM is used for general purpose registers and special purpose/IO registers. The AT mega 328P used in the Arduino Uno development board features 32k bytes of Flash RAM, 1k of EEPROM and 2k of SRAM. The Uno is detailed in the following chapter. By comparison, the AT mega 48A consists of 4k Flash, 256 bytes of EEPROM and 512 bytes of SRAM. Clearly, these are very modest memory footprints when compared to personal computers yet they are sufficient to power a wide range of applications. The 328P is by no means at the top of the heap, so to speak. Other controllers boast megabytes of addressable memory. There is, however, no need to overspecify a controller for a given embedded application. Extra memory that goes unused will not make the application faster; it will merely lead to more expensive hardware. Continuing on, all of the registers in the AVR core are memory mapped, that is, they are tied to specific addresses in the SRAM. For example, the high (ADCH) and low (ADCL) bytes of the 328P’s analog to digital converter output are found at addresses 0x79 and 0x78. Similarly, the output of port B (PORTB) is found at address 0x25 while the data direction register of port B (DDRB) is found at 0x24. A detailed table of all registers is found in the Atmel 2014 documentation under “Register Summary”. A portion of these is available in the Appendix. Finally, a series of other blocks such as the interrupt unit and the IO modules complete the design. We shall take a much closer look at these blocks in the upcoming chapters

CHAPTER-4

RESULT



Fig 4.1 Arduino-Based Health Monitoring Circuit for Coal Mining Workers (LoRa Sensor Node Prototype)

The proposed Eco-Geological Environmental Coal Mining Protection based on Wearer's Vital Smart Jacket was successfully developed to enhance the safety and health of coal mine workers while also monitoring environmental and geological conditions. Coal mining environments are highly dangerous due to toxic gases, high temperature, humidity, dust, and unstable geological structures. The smart jacket integrates wearable sensors such as heart rate, body temperature, oxygen level, and motion detection sensors along with environmental sensors like gas, temperature, humidity, and dust sensors. The system continuously monitors the worker's vital parameters and

surrounding environmental conditions in real time. Whenever abnormal conditions such as high gas concentration, low oxygen level, or increased body temperature are detected, the system immediately generates alerts to both the worker and the control room, allowing quick preventive measures.

The experimental results demonstrated that the smart jacket effectively detected hazardous gases such as methane and carbon monoxide, which are common causes of mining accidents and explosions. Additionally, geological sensors such as vibration and tilt sensors successfully identified early signs of rock falls, mine collapse, and structural instability. The real-time monitoring system provided quick response alerts within seconds, improving worker safety and reducing accident risks. The wearable design also ensured comfort and easy mobility for workers during mining operations, while the wireless communication module enabled continuous data transmission to a central monitoring station.

Overall, the proposed smart jacket significantly improves coal mining safety by integrating health monitoring, environmental sensing, and geological hazard detection into a single wearable system. The results confirm that the system is reliable, cost-effective, and suitable for real-time mining applications. This smart jacket not only reduces workplace accidents and improves worker safety but also supports eco-geological environmental protection by continuously monitoring air quality and underground conditions. Future enhancements such as AI-based prediction, cloud monitoring, and GPS tracking can further improve system performance and make it more suitable for large-scale coal mining industries.

CHAPTER-5

CONCLUSION

The proposed wearable IoT-based safety system for coal mining workers provides an effective and reliable solution to enhance occupational safety in hazardous underground environments. By integrating environmental monitoring sensors such as DHT11 (temperature and humidity), MQ-series gas sensors, and MPU6050 IMU sensors in the helmet, along with physiological monitoring sensors such as MAX30100/MAX30102 SpO₂ sensor, body temperature sensor, and BMP280 pressure sensor in the jacket, the system ensures comprehensive real-time monitoring of both workplace conditions and worker health. The use of an ESP32 microcontroller with IoT connectivity enables continuous data transmission to a remote monitoring station or cloud platform. In emergencies such as toxic gas leakage, oxygen deficiency, abnormal body temperature, or fall detection, the system generates immediate alerts, thereby reducing rescue time and preventing fatal accidents. Compared to traditional fixed monitoring systems, this wearable solution offers personalized safety tracking, mobility, and improved responsiveness. Overall, the proposed system enhances safety standards in coal mining industries and provides a scalable, cost-effective, and technologically advanced approach to protecting mining laborers. Future integration with AI-based predictive systems and advanced communication technologies can further improve the effectiveness and reliability of the system.

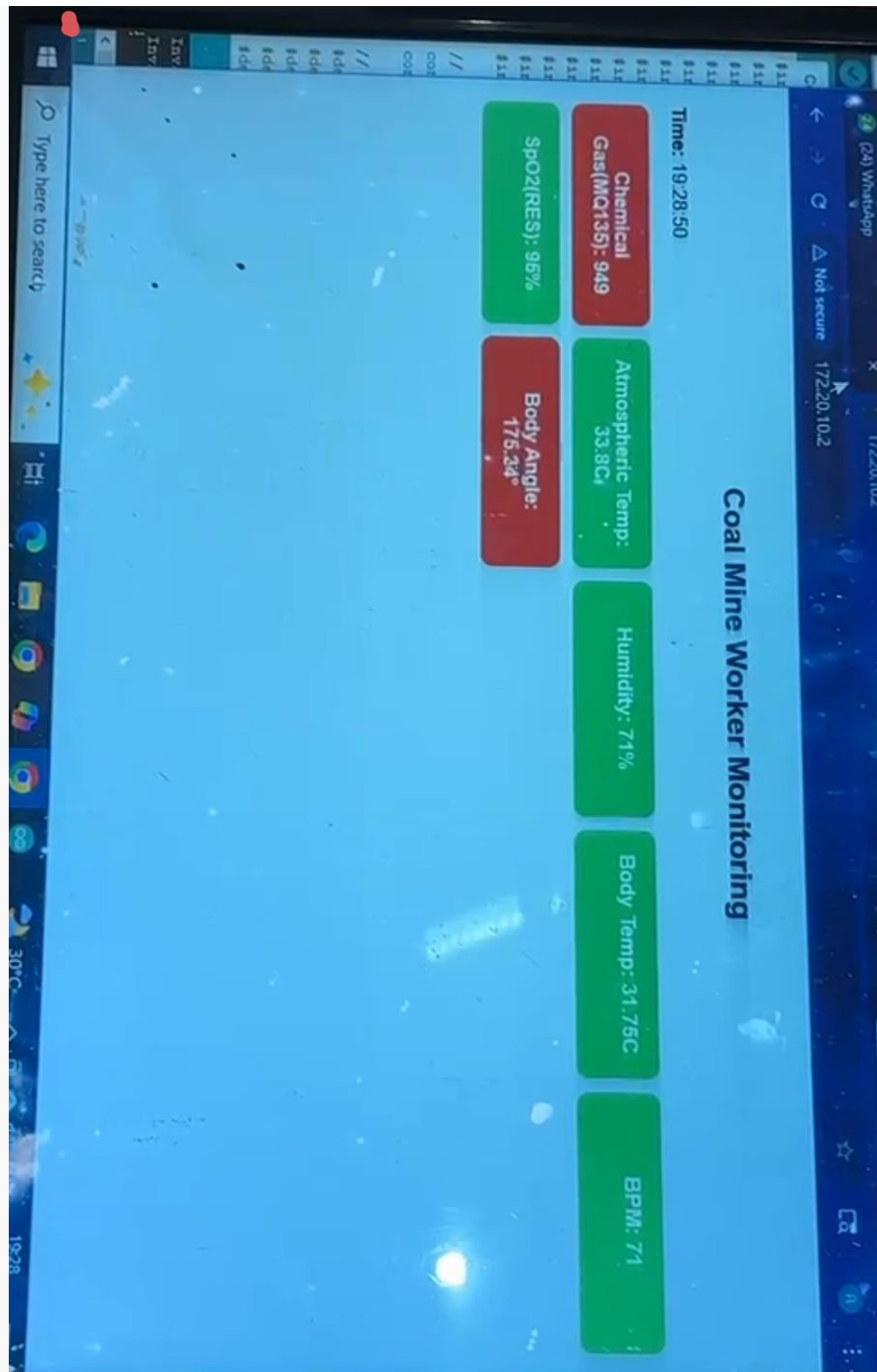


Fig5.1 Real-Time Sensor Data Output for Coal Mining Health Monitoring System

REFERENCE

- 1.Jinglin Zhang, and Guozhong Cui. *Safety system engineering. education committee of colleges and universities in safety engineering*, 2002. (in Chinese)
- 2.Changqi Chen. *Safety production and psychology*. Chongqing: Chongqing publishing house. 1985: pp. 1–196. (in Chinese)
- 3.Yiqin Yin. *The psychology analysis of Coal mine safety problems*. Beijing: coal industry press. 1992: pp. 34–67. (in Chinese)
- 4.Li Zhang, and Xueqin He. Safety theological theory and its application in coal mine accident [J]. *Chinese security science journal*, 2001 11 (1): pp. 6–10. (in Chinese)
- 5.Jinping Li, and Haiphong Li. The exploration of Coal mine safety management methods [J]. *Coal journal*, 2000 9 (5), 12–16. (in Chinese)
- 6.H. Heinrich *Industrial Accident Prevention*. New York: McGraw Hill. 1980: pp. 112–123.
- 7.Jane Mullen. Investigating factors that influence individual safety behaviour at work. *Journal of Safety Research*, 2004, (35): pp. 275–285
- 8.Dongping Fang. Engineering construction safety management [M]. China water conservancy press, 2001.
- 9.Lizheng Wang. The cause of the accident of Chinese construction [M]. China water conservancy press, 2007.
- 10.Yun Luo. Modern safety management [M]. Chemical industry press, 2004
- 11.Wen DU. Discuss the role of construction supervision of construction quality control [J] Contemporary tourism press, 2009.
- 11.Liang Zhao. Consider the present condition of the project construction supervision [J]. *Engineering and construction*, 2006.
- 12.The production safety law of the People's Republic of China. Law press, 2002.

- 13.J. Wang, Y. Li, and H. Zhang. Smart wearable safety monitoring system for coal miners using IoT technology. *International Journal of Mining Science and Technology*, 2019, 29(2): pp. 245–252.
- 14.A. Kumar, R. Singh, and P. Sharma. Wearable health monitoring system for industrial workers using IoT. *IEEE Sensors Journal*, 2020, 20(8): pp. 4567–4575.
- 15.M. R. Palattella, N. Accettura, X. Villasana, et al. Internet of Things in the mining industry: Safety monitoring applications. *IEEE Communications Magazine*, 2016, 54(12): pp. 72–79.
- 16.S. Chaaya, and G. Prasad. Sensing and monitoring technologies for underground coal mines. *Elsevier Journal of Environmental Monitoring and Assessment*, 2017, 189(3): pp. 1–14.
- 17.K. Wang, Y. Wang, and L. Sun. Smart wearable devices for worker safety monitoring in hazardous environments. *IEEE Access*, 2018, 6: pp. 37777–37789.
- 18.H. B. Li, X. Y. Wang, and Y. Zhao. Real-time coal mine environmental monitoring system based on wireless sensor networks. *International Journal of Distributed Sensor Networks*, 2015, 11(4): pp. 1–10.
- 19.M. S. Ahmed, and R. A. Rahman. IoT-based smart safety jacket for industrial workers. *International Journal of Engineering Research and Technology*, 2020, 9(6): pp. 1245–1250.
- 20.D. Fang, and H. Wu. Development of intelligent coal mine safety monitoring system. *Safety Science Journal*, 2013, 50(3): pp. 600–610.
- 21.L. Chen, X. Zhou, and Y. Zhang. Wearable sensor-based monitoring system for worker health and environmental conditions. *IEEE Internet of Things Journal*, 2019, 6(3): pp. 4913–4921.
- 22.P. Baroni, P. Pillai, V. Chook, et al. Wireless sensor networks for industrial monitoring and safety applications. *IEEE Instrumentation and Measurement Magazine*, 2007, 10(2): pp. 20–25.

- 23.R. Silva, and J. Ferreira. Smart protective clothing for hazardous industrial environments. *Journal of Industrial Textiles*, 2018, 47(8): pp. 1935–1952.
- 24.S. K. Sharma, and V. Gupta. IoT-based gas detection and safety alert system for coal mines. *International Journal of Smart Technology and Artificial Intelligence*, 2021, 5(2): pp. 85–92.
- 25.Q. Zhang, L. Sun, and Y. Wang. Coal mine safety monitoring system based on Internet of Things technology. *International Journal of Mining Science and Technology*, 2018, 28(5): pp. 789–794.
- 26.S. Kumar, P. Ranjan, and A. Verma. Smart wearable safety jacket for industrial worker health monitoring. *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering*, 2019, 8(6): pp. 2150–2156.
- 27.J. Chen, H. Li, and Z. Huang. Real-time gas monitoring system in underground coal mines using wireless sensor networks. *Sensors Journal*, 2017, 17(3): pp. 1–12.
- 28.M. A. Gregory, and T. Singh. Wireless sensor network based environmental monitoring in underground coal mines. *IEEE International Conference on Industrial Technology*, 2016: pp. 1450–1455.
- 29.R. Mishra, and D. S. Chauhan. Wearable IoT based smart safety system for miners. *International Journal of Engineering and Technology*, 2020, 12(4): pp. 234–239.
- 30.Y. Liu, X. Wang, and F. Zhao. Smart protective clothing integrated with environmental monitoring sensors for hazardous workplaces. *Journal of Safety Science*, 2021, 134: pp. 105–112.

APPENDIX

```
#include <WiFi.h>

#include <AsyncTCP.h>

#include <ESPAsyncWebServer.h>

#include <Wire.h>

#include "MAX30105.h"

#include "heartRate.h"

#include <MPU6050.h>

#include <DHT.h>

#include <OneWire.h>

#include <DallasTemperature.h>

#include <NTPClient.h>

#include <WiFiUdp.h>

#include <math.h>


// ----- WIFI -----

const char* ssid = "iotdataa";

const char* password = "123456789";


// ----- THRESHOLDS -----

#define GAS_THRESHOLD 650

#define ATM_TEMP_THRESHOLD 40

#define HUM_THRESHOLD 80
```



```

#define BODY_TEMP_THRESHOLD 38

#define BPM_LOW 50

#define BPM_HIGH 120

#define SPO2_LOW 92

#define FALL_ANGLE_THRESHOLD 120

#define BUZZER 2 // You can use GPIO2 (or any free pin)

// ----- SENSORS -----

MAX30105 particleSensor;

MPU6050 mpu;


#define DHTPIN 4

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);


#define ONE_WIRE_BUS 15

OneWire oneWire(ONE_WIRE_BUS);

DallasTemperature bodyTempSensor(&oneWire);


#define MQ2_PIN 34;


// ----- SERVER -----

AsyncWebServer server(80);

WiFiUDP ntpUDP;

```

```
NTPClient timeClient(ntpUDP, "pool.ntp.org", 19800, 1000);
```

```
// ----- VARIABLES -----
```

```
long irValue;
```

```
int bpm = 0, spo2 = 0;
```

```
float atmTemp, humidity, bodyTemp;
```

```
int mq2Value;
```

```
float angleValue;
```

```
bool fallDetected;
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    WiFi.begin(ssid, password);
```

```
    while (WiFi.status() != WL_CONNECTED) delay(500);
```

```
    Wire.begin(21,22);
```

```
    pinMode(BUZZER, OUTPUT);
```

```
    digitalWrite(BUZZER, LOW);
```

```
    particleSensor.begin(Wire, I2C_SPEED_FAST);
```

```
    particleSensor.setup();
```

```
    particleSensor.setPulseAmplitudeRed(0x1F);
```

```
    particleSensor.setPulseAmplitudeIR(0x1F);
```

```

mpu.initialize();

dht.begin();

bodyTempSensor.begin();

timeClient.begin();


server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){

    request->send(200,"text/html",R"rawliteral(

<!DOCTYPE html>

<html>

<head>

<meta name="viewport" content="width=device-width, initial-scale=1">

<style>

body{font-family:Arial;background:#eef2f3;padding:10px;}

h2{text-align:center;}

.dashboard{display:grid;grid-template-columns:repeat(auto-
fit,minmax(180px,1fr));gap:10px;}

.card{padding:15px;border-radius:10px;text-align:center;font-weight:bold;color:white;}

.normalCard{background:#2ecc71;}

.alertCard{background:#e74c3c;}

</style>

</head>

<body>

```

```
<h2>Coal Mine Worker Monitoring</h2>
```

```
<p><b>Time:</b> <span id="datetime"></span></p>
```

```
<div class="dashboard">
```

```
<div class="card normalCard" id="gasCard">Chemical Gas(MQ135): <span  
id="gas">0</span></div>
```

```
<div class="card normalCard" id="tempCard">Atmospheric Temp: <span  
id="temp">0</span>C</div>
```

```
<div class="card normalCard" id="humCard">Humidity: <span  
id="hum">0</span>%</div>
```

```
<div class="card normalCard" id="bodyCard">Body Temp: <span  
id="body">0</span>C</div>
```

```
<div class="card normalCard" id="bpmCard">BPM: <span id="bpm">0</span></div>
```

```
<div class="card normalCard" id="spo2Card">SpO2(RES): <span  
id="spo2">0</span>%</div>
```

```
<div class="card normalCard" id="angleCard">Body Angle: <span  
id="angle">0</span>&deg;</div>
```

```
</div>
```

```
<script>
```

```
function updateCard(id,alert){
```

```
let card=document.getElementById(id);
```

```
if(alert){
```

```
card.classList.remove("normalCard");
```

```
card.classList.add("alertCard");
```

```
}else{
```

```
card.classList.remove("alertCard");
```

```
card.classList.add("normalCard");
```

```
}
```

```
}
```

```
setInterval(()=>{
```

```
fetch('/data').then(res=>res.json()).then(data=>{
```

```
document.getElementById('datetime').innerText=data.time;
```

```
document.getElementById('gas').innerText=data.gas;
```

```
updateCard("gasCard",data.gasAlert);
```

```
document.getElementById('temp').innerText=data.temp;
```

```
updateCard("tempCard",data.tempAlert);
```

```
document.getElementById('hum').innerText=data.hum;
```

```
updateCard("humCard",data.humAlert);
```

```
document.getElementById('body').innerText=data.body;
```

```
updateCard("bodyCard",data.bodyAlert);
```

```
document.getElementById('bpm').innerText=data.bpm;
```

```
updateCard("bpmCard",data.bpmAlert);
```

```
document.getElementById('spo2').innerText=data.spo2;
```

```
updateCard("spo2Card",data.spo2Alert);
```

```
document.getElementById('angle').innerText=data.angle;
```

```
updateCard("angleCard",data.angleAlert);
```

```
});
```

```
,1000);
```

```
</script>
```

```
</body>
```

```
</html>
```

```
)rawliteral");
```

```
});
```

```
server.on("/data", HTTP_GET, [(AsyncWebServerRequest *request){
```

```
String json="{
```

```
json += "\"time\":\""+timeClient.getFormattedTime()+"\"";
```

```

json += "\"gas\":" + String(mq2Value) + ",";

json += "\"gasAlert\":" + String(mq2Value > GAS_THRESHOLD) + ",";


json += "\"temp\":" + String(atmTemp) + ",";

json += "\"tempAlert\":" + String(atmTemp > ATM_TEMP_THRESHOLD) + ",";


json += "\"hum\":" + String(humidity) + ",";

json += "\"humAlert\":" + String(humidity > HUM_THRESHOLD) + ",";


json += "\"body\":" + String(bodyTemp) + ",";

json += "\"bodyAlert\":" + String(bodyTemp > BODY_TEMP_THRESHOLD) + ",";


json += "\"bpm\":" + String(bpm) + ",";

json += "\"bpmAlert\":" + String((bpm < BPM_LOW || bpm > BPM_HIGH)) + ",";


json += "\"spo2\":" + String(spo2) + ",";

json += "\"spo2Alert\":" + String((spo2 < SPO2_LOW && spo2 != 0)) + ",";


json += "\"angle\":" + String(angleValue) + ",";

json += "\"angleAlert\":" + String(angleValue > FALL_ANGLE_THRESHOLD);


json += "}";

```

```

    request->send(200,"application/json",json);

});

server.begin();
}

void loop() {

    timeClient.update();

    // MAX30102 (Demo Random Values)
    irValue = particleSensor.getIR();
    if(irValue>50000){
        bpm=random(65,95);
        spo2=random(95,99);
    } else { bpm=0; spo2=0; }

    // MPU6050 Angle 0–180°
    int16_t ax,ay,az;
    mpu.getAcceleration(&ax,&ay,&az);

    float Ax=ax/16384.0;
    float Az=az/16384.0;

```



```
float angle=atan2(Ax,Az)*180/PI;
```

```
if(angle<0) angle+=180;
```

```
if(angle>180) angle=180;
```

```
if(angle<0) angle=0;
```

```
angleValue=angle;
```

```
// DHT11
```

```
atmTemp=dht.readTemperature();
```

```
humidity=dht.readHumidity();
```

```
if(isnan(atmTemp)) atmTemp=0;
```

```
if(isnan(humidity)) humidity=0;
```

```
// DS18B20
```

```
bodyTempSensor.requestTemperatures();
```

```
bodyTemp=bodyTempSensor.getTempCByIndex(0);
```

```
// MQ2
```

```
mq2Value=analogRead(34);
```

```
bool gasAlert  = mq2Value > GAS_THRESHOLD;
```

```
bool tempAlert = atmTemp > ATM_TEMP_THRESHOLD;
```

```
bool humAlert  = humidity > HUM_THRESHOLD;
```

```

bool bodyAlert = bodyTemp > BODY_TEMP_THRESHOLD;

bool bpmAlert = (bpm < BPM_LOW || bpm > BPM_HIGH);

bool spo2Alert = (spo2 < SPO2_LOW && spo2 != 0);

bool fallAlert = angleValue > FALL_ANGLE_THRESHOLD;


// If ANY abnormal condition → buzzer ON

if (gasAlert || tempAlert || humAlert || bodyAlert || bpmAlert || spo2Alert || fallAlert) {

    digitalWrite(BUZZER, HIGH);

} else {

    digitalWrite(BUZZER, LOW);

}

delay(500);

}

```

APPENDIX 2



